Machine Learning — Tutorial: Complete Solutions

Logistic Regression · GDA · Naïve Bayes · PCA · K-Means

Decision Trees · Ensemble Methods · Metrics · Gradient Descent

20 Questions | Detailed Step-by-Step | Beginner Friendly | A4 Printable

Quick-Reference Formulas

Sigmoid: $\sigma(z)=1/(1+e^{-z})$ | GD: $\theta\leftarrow\theta-\eta\nabla L$ | Bayes: $P(y|x)\propto P(x|y)P(y)$ | Laplace: $\hat{P}=(\text{count}+1)/(N+k)$

Gini: $1-\Sigma p_i^2$ | Entropy: $-\Sigma p_i \log_2 p_i$ | $F1=2PR/(P+R)$ | PCA score: $z=u^T x$ | $SSE=\Sigma(y_i-\bar{y})^2$

Q1 — Discriminative (Logistic Regression) vs Generative (GDA)

Explain the distinction. When does GDA outperform logistic regression and vice versa?

? Discriminative — Logistic Regression

Models P(y|x) *directly* — the probability of the label given the features. Does not model how x is distributed.

$$P(y=1|x;\ \theta) = \sigma(\theta^T x)$$

No assumptions about the shape of P(x). Directly optimises the classification boundary.

? Generative — GDA

Models the joint distribution P(x,y) = P(x|y)P(y). Assumes $x|y=k \sim N(\mu_k,\Sigma)$. Then uses Bayes' theorem to find P(y|x).

$$P(y|x) \propto P(x|y)\cdot P(y)$$

Knows how data was generated — more information, more assumptions.

GDA outperforms when:

- Gaussian assumption holds (data truly normal)
- Training data is small — GDA uses data more efficiently (generative models extract more info per sample)
- Both classes follow Gaussian distributions with equal covariance

Logistic Regression preferred when:

- Data is not Gaussian (e.g. binary, sparse, bag-of-words)
- Large training set available
- Robust to model mis-specification — fewer assumptions violated
- Goal is purely accurate classification, not data generation

---

Q2 — Naïve Bayes Zero-Probability Problem & Laplace Smoothing

'offer' never appears in legitimate emails. What happens when a legitimate email with 'offer' arrives at test time?

? What Goes Wrong

1	**Training observation:** P(word='offer' | y=0) = $0/n_0$ = 0. Zero count → zero probability estimate (MLE).
2	**At test time**, Naïve Bayes computes: `P(y=0 | email) ∝ P(y=0) · ∏ P(wᵢ | y=0)` The term P('offer' | y=0) = 0 makes the **entire product = 0**, regardless of all other evidence.
3	**Consequence:** The classifier always assigns P(y=0|email)=0 whenever 'offer' appears, even if the rest of the email is clearly legitimate. This is called the **zero-probability (sparse data) problem**.

? Laplace Smoothing Fix

Add a pseudo-count of 1 to every word count, and add |V| (vocabulary size) to the denominator:

```
P̂(word='offer' | y=k) = (count(offer, y=k) + 1) / (total_words_in_class_k +
|V|)
```

General smoothed formula:

```
P̂(xⱼ=v | y=k) = (count(xⱼ=v, y=k) + α) / (Nₖ + α·|values of xⱼ|)
```

where α=1 for Laplace smoothing. Now every word has a small but non-zero probability → no more zero products. The estimate is slightly biased toward uniformity but never zero.

Q3 — Logistic Regression: Linear Boundary & Feature Engineering

Is it true that logistic regression can ONLY produce a straight-line decision boundary? How can non-linear boundaries be achieved?

The claim is PARTLY correct:

In the **original feature space**, logistic regression produces a **linear decision boundary**:

```
θᵀx = 0 → straight line (2D) /
hyperplane (nD)
```

The prediction is h=σ(θ^Tx), and the boundary is where θ^Tx=0 — always a linear function of the raw features.

Standard Fix: Feature Engineering

Add polynomial / interaction features **before** fitting. The classifier itself doesn't change — only the input does.

```
Original: x = [x₁, x₂]
```

```
Extended: φ(x) = [x₁, x₂, x₁²,
x₂², x₁x₂]
```

*Logistic regression on φ(x) produces a **non-linear boundary in the original x-space** (e.g. circles, ellipses).*

? Claim is correct in raw feature space. Fix: map x→φ(x) with polynomial features. Logistic regression on φ(x) captures non-linear boundaries.

Q4 — Naïve Bayes: Compute P(y=1 | x_1=1, x_2=0)

x_1	x_2	y

1	1	1
1	0	0
0	1	0
0	0	0

1 **Class priors:** 1 example has y=1; 3 have y=0. Total = 4.

$P(y=1) = 1/4 = 0.25$ $P(y=0) = 3/4 = 0.75$

2 **Likelihoods for y=1 (only 1 example: $x_1=1, x_2=1$):**

$P(x_1=1|y=1) = 1/1 = 1.0$

$P(x_2=0|y=1) = 0/1 = 0.0$

Problem: $P(x_2=0|y=1)=0$ → apply Laplace smoothing (add-1)

$P(x_2=0|y=1) = (0+1)/(1+2) = 1/3$ $P(x_2=1|y=1) = (1+1)/(1+2) = 2/3$

3 **Likelihoods for y=0 (3 examples: (1,0),(0,1),(0,0)):**

$P(x_1=1|y=0) = 1/3$ $P(x_2=0|y=0) = 2/3$

(No zero counts here, smoothing optional)

4 **Unnormalised posteriors (∝):**

$P(y=1|x_1=1,x_2=0) \propto P(x_1=1|y=1) \cdot P(x_2=0|y=1) \cdot P(y=1)$

$= 1.0 \times (1/3) \times 0.25 = 0.0833$

$P(y=0|x_1=1,x_2=0) \propto (1/3) \times (2/3) \times 0.75 = 0.1667$

5 **Normalise:** Total = 0.0833 + 0.1667 = 0.2500

$P(y=1|x_1=1,x_2=0) = 0.0833/0.25 = 0.333$

$P(y=0|x_1=1,x_2=0) = 0.1667/0.25 = 0.667$

? $P(y=1|x_1=1,x_2=0) \approx 0.333$ → Predict y = 0 (with Laplace smoothing)

Q5 — PCA: Projection Score, Reconstruction & Error

$x=[4,2]^T$, $u_1=(1/\sqrt{5})[2,1]^T$. Find (a) score z, (b) $\hat{x}$, (c) $\|x-\hat{x}\|^2$.

a **Projection score $z = u_1^T x$:**

$u_1 = (1/\sqrt{5})[2,1]^T = [2/\sqrt{5}, 1/\sqrt{5}]^T$

$z = u_1^T x = (2/\sqrt{5}) \cdot 4 + (1/\sqrt{5}) \cdot 2 = 8/\sqrt{5} + 2/\sqrt{5} = 10/\sqrt{5} = \sqrt{5} \cdot 2 = 2\sqrt{5}$

$z = 10/\sqrt{5} = 10/2.2361 \approx 4.4721$

b **Reconstructed point $\hat{x} = z \cdot u_1$:**

$\hat{x} = z \cdot u_1 = (10/\sqrt{5}) \cdot (1/\sqrt{5})[2,1]^T = (10/5)[2,1]^T = 2[2,1]^T = [4, 2]^T$

Interesting: $\hat{x} = x$ exactly! This means x lies exactly on the PC1 axis.

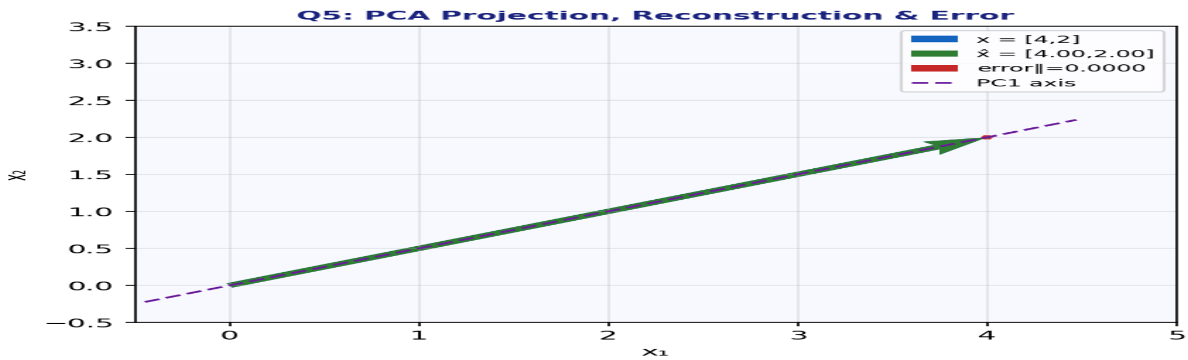
c **Reconstruction error $\|x - \hat{x}\|^2$:**

$x - \hat{x} = [4,2]^T - [4,2]^T = [0,0]^T$

$\|x - \hat{x}\|^2 = 0^2 + 0^2 = 0$

Zero error! The point x=[4,2] lies perfectly on the first PC direction $u_1=[2,1]/\sqrt{5}$.

? z = 2√5 ≈ 4.472 | $\hat{x}$ = [4, 2]^T | Reconstruction error = 0 (x lies on PC1)



Blue arrow = x. Green arrow = $\hat{x}$ (reconstructed). They overlap completely because x lies exactly on the PC1 axis — zero reconstruction error.

Q6 — Two Failure Modes of K-Means

? Failure Mode 1: Bad Initialisation

K-means converges to a **local minimum**, not the global optimum, depending on where centroids are initialised. Two centroids starting in the same cluster leave another cluster unrepresented.

Remedy: K-Means++ Initialisation

Choose first centroid randomly. Each subsequent centroid is chosen with probability proportional to its squared distance from the nearest existing centroid. This spreads centroids out, dramatically reducing bad initialisations.

? Failure Mode 2: Wrong K / Non-Spherical Clusters

K-means assumes clusters are roughly spherical and equally sized (uses Euclidean distance). It fails on elongated, ring-shaped, or clusters of very different sizes/densities.

Remedy: DBSCAN or Gaussian Mixture Models

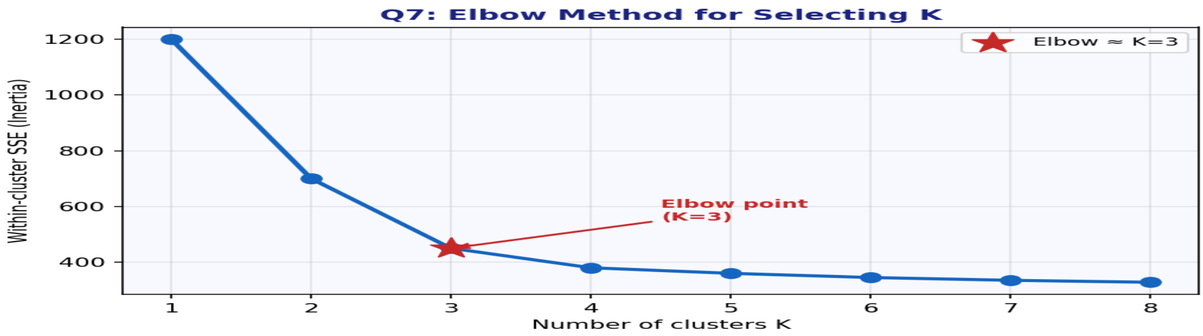
DBSCAN finds clusters of arbitrary shape using density. GMMs (soft K-means) allow elliptical clusters via a covariance matrix per cluster.

Q7 — Elbow Method for Selecting K in K-Means

1 **What is plotted:** The **Within-Cluster Sum of Squared Errors (SSE)** (also called inertia) on the y-axis against the number of clusters K on the x-axis.

2 **Why SSE always decreases:** More clusters → each cluster is smaller → points are closer to their centroid → squared distances shrink. At K=n, every point is its own cluster and SSE=0.

3	**What the elbow indicates:** Beyond the elbow, adding more clusters gives only marginal improvement in SSE — the curve flattens. The elbow is the **optimal K**: it captures most structure without over-clustering.
4	**Limitation:** The elbow is sometimes ambiguous — the curve may decrease smoothly with no clear bend. In such cases, use the **Silhouette score** as a complementary method.



Inertia (SSE) always decreases with K. The elbow at K=3 suggests 3 clusters is the best tradeoff between compression and fit.

Q8 — Imbalanced Data: Why Accuracy Misleads & Better Metrics

95% negative, 5% positive. Classifier predicts negative always → 95% accuracy. Why is this misleading?

1 **Why accuracy is misleading:**
A trivial classifier that never outputs 'Positive' achieves 95% accuracy by exploiting the class imbalance. It has learned nothing useful — it completely **ignores the minority class** (the one we care about most).

2 **Metric 1 — Precision:** Of all predicted positives, how many are correct?

```
Precision = TP / (TP + FP)
```

For our trivial classifier: TP=0, FP=0 → Precision = undefined (or 0).
Measures: are the positive predictions trustworthy?

3 **Metric 2 — Recall (Sensitivity / TPR):** Of all actual positives, how many did we catch?

```
Recall = TP / (TP + FN)
```

For trivial classifier: TP=0, FN=all_positives → Recall = 0. Exposes the failure.
Measures: did we miss any real positives?

4 **Bonus — F1 Score** combines both:

```
F1 = 2 · Precision · Recall / (Precision + Recall)
```

F1 = 0 for the trivial classifier, correctly revealing it is useless.

? Use Precision=TP/(TP+FP) and Recall=TP/(TP+FN). Both are 0 for trivial classifier.

Q9 — PCA Variance Explained: 200-Feature Dataset

λ_1=60, λ_2=40, λ_3=25, λ_4=15, remaining 196 eigenvalues sum to 60.

a **Total variance = sum of ALL eigenvalues:**

Total = $\lambda_1+\lambda_2+\lambda_3+\lambda_4$ + Σ_remaining = 60+40+25+15+60 = 200

b **Fraction explained by first 3 PCs:**

$\Sigma(\lambda_1..\lambda_3)$ / Total = (60+40+25) / 200 = 125/200 = 0.625 = 62.5%

c **PCs needed to explain ≥70% of variance:**

1 PC: 60/200 = 30.0%

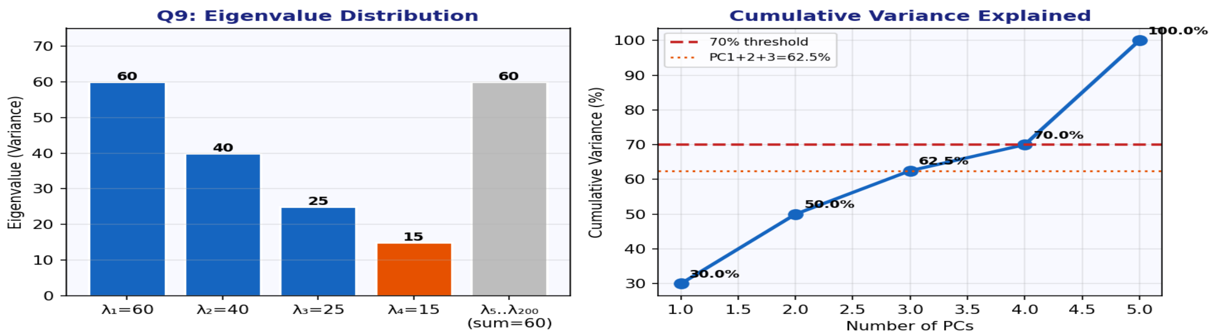
2 PCs: 100/200 = 50.0%

3 PCs: 125/200 = 62.5%

4 PCs: 140/200 = 70.0% ✓ ← first to reach 70%

Need **4 components** to explain at least 70%.

? (a) Total = 200 | (b) 3 PCs explain 62.5% | (c) Need 4 PCs for ≥70%



Left: eigenvalue bar chart. Right: cumulative variance — 4 PCs reach exactly 70%.

Q10 — K-Means: One Complete Iteration on {3,5,11,13}

K=2, initial centroids μ_1=4, μ_2=12. Perform assignment then update.

1 **Assignment step** — assign each point to nearest centroid:

Point 3: |3−4|=1 < |3−12|=9 → Cluster 1

Point 5: |5−4|=1 < |5−12|=7 → Cluster 1

Point 11: |11−4|=7 > |11−12|=1 → Cluster 2

Point 13: |13−4|=9 > |13−12|=1 → Cluster 2

2 **Update step** — recompute centroids as cluster means:

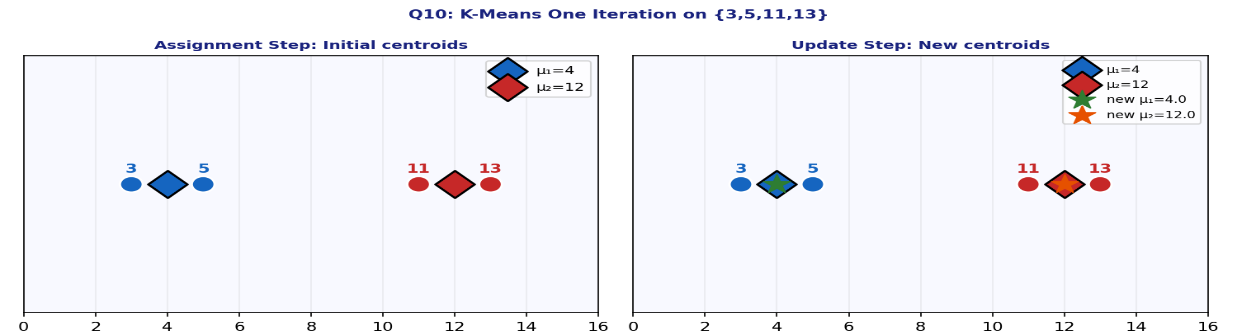
μ_1_new = mean(3, 5) = 8/2 = 4

μ_2_new = mean(11, 13) = 24/2 = 12

3 **Convergence check:** μ_1_new = 4 = μ_1_old and μ_2_new = 12 = μ_2_old.

The centroids did not change → Algorithm has converged after just one iteration.

? Clusters: C_1={3,5} μ=4, C_2={11,13} μ=12. Converged — centroids unchanged.



Assignment step (left): points coloured by nearest centroid. Update step (right): star markers = new centroids (unchanged → converged).

Q11 — Decision Tree Gini Impurity & Information Gain

Parent: 7A + 5B (n=12). Split: Left=6A+0B, Right=1A+5B.

a **Gini impurity of parent node:**

$$\text{Gini} = 1 - \Sigma\, p_i^2 = 1 - [(7/12)^2 + (5/12)^2]$$
$$= 1 - [0.3403 + 0.1736] = 1 - 0.5139 = 0.4861$$

b **Gini of left child (6A, 0B — pure!):**

$$\text{Gini\_L} = 1 - [(6/6)^2 + (0/6)^2] = 1 - 1 = 0.000$$

Gini of right child (1A, 5B):

$$\text{Gini\_R} = 1 - [(1/6)^2 + (5/6)^2] = 1 - [0.0278+0.6944] = 0.2778$$

Weighted average after split:

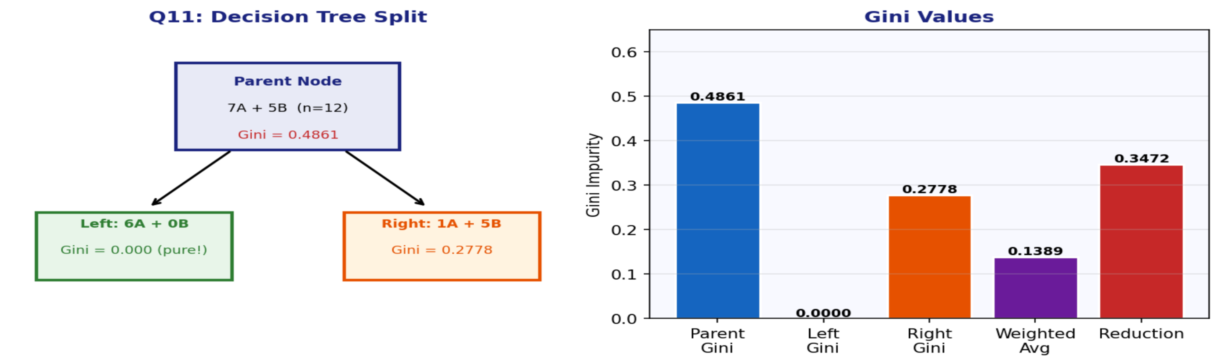
$$\text{Gini\_split} = (6/12)\cdot 0 + (6/12)\cdot 0.2778 = 0 + 0.1389 = 0.1389$$

c **Reduction in impurity (Information Gain):**

$$\Delta\text{Gini} = \text{Gini\_parent} - \text{Gini\_split} = 0.4861 - 0.1389 = 0.3472$$

This is a **very good split**: nearly 35% reduction in impurity. The left child is completely pure (0.0), and the right is much more pure than the parent.

? (a) 0.4861 (b) Weighted Gini=0.1389 (c) Reduction=0.3472 — excellent split



Left: decision tree diagram. Right: bar chart showing Gini values at each node. Left child is perfectly pure (Gini=0).

Q12 — Decision Tree Overfitting: Pre-Pruning & Post-Pruning

100% train accuracy, 60% test accuracy. Name the phenomenon. Describe one pre- and one post-growth strategy.

This phenomenon is called Overfitting (also: over-growing the tree).

? Strategy 1: Early Stopping (Pre-growth)

Stop growing the tree **before** it reaches full depth by imposing constraints:

- **max_depth**: limit tree depth (e.g. depth ≤ 5)
- **min_samples_split**: don't split if node has < k examples
- **min_impurity_decrease**: only split if ΔGini > threshold

Prevents memorisation of rare training patterns. Faster and simpler than post-pruning.

? Strategy 2: Cost-Complexity Pruning (Post-growth)

Grow the full tree first, then **prune subtrees** that don't justify their complexity:

- **Reduced-Error Pruning**: remove nodes that don't hurt validation accuracy
- **Cost-Complexity Pruning (α)**: penalise tree size via loss = train_error + α·(num_leaves)

Cross-validate α to find the best tradeoff between fit and simplicity.

Q13 — GDA: Closed-Form MLE for φ, μ_0, μ_1, Σ

State the four maximum-likelihood estimates for GDA with shared Σ. n total examples, n_0 in class 0, n_1 in class 1.

The four closed-form MLEs:

```
1. φ = n₁/n
2. μ₀ = (1/n₀) · Σ_{i:yᵢ=0} xᵢ
3. μ₁ = (1/n₁) · Σ_{i:yᵢ=1} xᵢ
4. Σ = (1/n) · Σᵢ (xᵢ − μ_{yᵢ})(xᵢ − μ_{yᵢ})ᵀ
```

In plain words:

φ = fraction of training examples belonging to class 1.

μ_0 = empirical mean of all class-0 training points.

μ_1 = empirical mean of all class-1 training points.

Σ = pooled sample covariance — average outer product of each point's deviation from its class mean. Note the shared Σ assumes equal covariance for both classes.

Q14 — F1 Score & Medical Diagnosis Trade-offs

Classifier A: P=0.80, R=0.60. Classifier B: P=0.65, R=0.85. Compute F1 for each. Which is better for medical diagnosis?

A	**F1 for Classifier A:**
	`F1_A = 2·P·R/(P+R) = 2·0.80·0.60/(0.80+0.60)`
	`= 2·0.48/1.40 = 0.96/1.40 ≈ 0.6857`

B	**F1 for Classifier B:**
	`F1_B = 2·0.65·0.85/(0.65+0.85)`
	`= 2·0.5525/1.50 = 1.105/1.50 ≈ 0.7367`

?	**Medical diagnosis — which is better?** In medical diagnosis, **missing a positive case (False Negative) is very costly** — an undetected disease can be fatal. Therefore, **Recall** is the priority metric. Classifier B has Recall=0.85 vs Classifier A's 0.60 → **Classifier B is preferred**. Classifier B also has higher F1 (0.737 vs 0.686), confirming it's the better choice.

? F1_A≈0.686, F1_B≈0.737. Prefer Classifier B — higher Recall (0.85 vs 0.60) catches more true positives, critical in medical diagnosis.

Q15 — Logistic Regression: GD Update Rule & Why Cross-Entropy

State the gradient descent update rule for logistic regression. Why prefer cross-entropy loss over squared error?

? Gradient Descent Update Rule

For each parameter θ_j ($j = 0, 1, ..., d$):

$$\theta_j \leftarrow \theta_j - \eta \cdot (1/m) \cdot \Sigma_i \, (h\theta(x_i) - y_i) \cdot x_{ij}$$

Symbol definitions:

θ_j = the j-th parameter (weight)

η (eta) = learning rate — step size for each update

m = number of training examples

$h\theta(x_i) = \sigma(\theta^T x_i)$ = predicted probability for example i

y_i = true label (0 or 1) for example i

x_{ij} = the j-th feature of training example i

$(h\theta(x_i)-y_i)$ = prediction error (residual)

? Why Cross-Entropy, Not Squared Error?

? Squared Error on logistic output:

$$L = \tfrac{1}{2}(\sigma(\theta^T x) - y)^2$$

This creates a **non-convex loss surface** with many local minima — gradient descent may get stuck.

*Also, gradients become very small when predictions are near 0 or 1 (sigmoid saturation → **vanishing gradients**).*

? Cross-Entropy (Log-Loss):

$$L = -[y \cdot \log(h) + (1-y) \cdot \log(1-h)]$$

Derived from the **likelihood function** of the Bernoulli model — it is the natural loss for binary classification.

*Creates a **convex loss surface** → unique global minimum → gradient descent always converges. Large errors get large gradients, small errors get small gradients — ideal learning dynamics.*

Q16 — PCA: Two Criteria for Choosing k & Practical Consideration

Describe two criteria for choosing how many PCA components to retain, and one reason to choose fewer than the criteria suggest.

Criterion 1: Explained Variance Threshold

Choose the smallest k such that the retained PCs explain ≥ some threshold (e.g. 90%, 95%) of total variance:

$$\Sigma_{i=1}^{k} \lambda_i \; / \; \Sigma_{i=1}^{p} \lambda_i \geq \text{threshold}$$

Simple and widely used. Threshold choice is problem-dependent.

Criterion 2: Elbow / Scree Plot

Plot eigenvalues in decreasing order. Choose k at the 'elbow' — the point where the curve bends sharply and further PCs add little variance.

Complementary to the threshold method. Works well when there is a clear spectral gap in eigenvalues.

Practical reason to choose smaller k:

Even if 10 components explain 95% of variance, you might choose k=3 if that is enough for your downstream task (e.g. visualisation, classification) and you need fast inference, low memory, or easy human interpretability. Also, retaining too many components can reintroduce noise from small-variance components.

Q17 — GDA vs Logistic Regression: Asymptotic Performance

If GDA assumptions hold exactly: which achieves lower asymptotic error? What if they don't hold?

1. **If GDA assumptions hold exactly** (i.e. class-conditionals are truly Gaussian with shared Σ):
GDA is a **more efficient estimator** — it achieves lower asymptotic error because it uses additional structural information (the Gaussian form). It extracts more information per training example.
With small n, GDA converges faster to the Bayes-optimal classifier.

2	**Formal relationship:** When GDA assumptions hold, the GDA posterior P(y|x) is logistic in form — so GDA is strictly more powerful (it has a larger hypothesis class implicitly). Logistic regression can represent any posterior expressible by GDA, but GDA additionally constrains the class-conditional densities.
3	**When GDA assumptions do NOT hold** (e.g. bag-of-words, binary/discrete features, heavy-tailed data): GDA suffers from model mis-specification — its Gaussian assumption is wrong, introducing systematic error. **Logistic regression**, making no distributional assumption, performs better. It converges to the correct decision boundary regardless of P(x|y).

? GDA assumptions hold → GDA wins (asymptotically better, needs less data). Assumptions violated → Logistic Regression wins (robust, no distributional assumption).

Q18 — Bootstrap Sampling: Out-of-Bag (OOB) Examples

n=6 points, bootstrap sample of size 6 with replacement. Find P(point excluded) and OOB fraction.

a	**P(specific point $x^{(i)}$ is NOT in one draw):** `P(not picked in one draw) = (n-1)/n = 5/6` **P(not picked in ANY of n draws):** $P(x^{(i)}$ not in bootstrap sample$) = (1 - 1/n)^n = (5/6)^6$ $= (0.8333)^6 \approx 0.3349 \approx 33.5\%$
b	**As n → ∞:** $(1 - 1/n)^n \to e^{-1} \approx 0.368$ (about 36.8%) So **roughly 36.8% of original examples are excluded** on average from each bootstrap sample.
c	**These excluded points are called 'Out-of-Bag (OOB) examples'.** They are used as a **free validation set**: since each tree in a Random Forest was trained without seeing its OOB examples, we can evaluate it on them to get an unbiased estimate of test error — without needing a separate validation set.

? P(excluded)=$(5/6)^6$≈33.5% (→36.8% as n→∞). OOB examples = free validation set.

Q19 — Regression Tree: SSE Before & After Split at x=5

Points: (1,3),(2,5),(3,4),(8,9),(9,11). Split at x=5: left x≤5, right x>5.

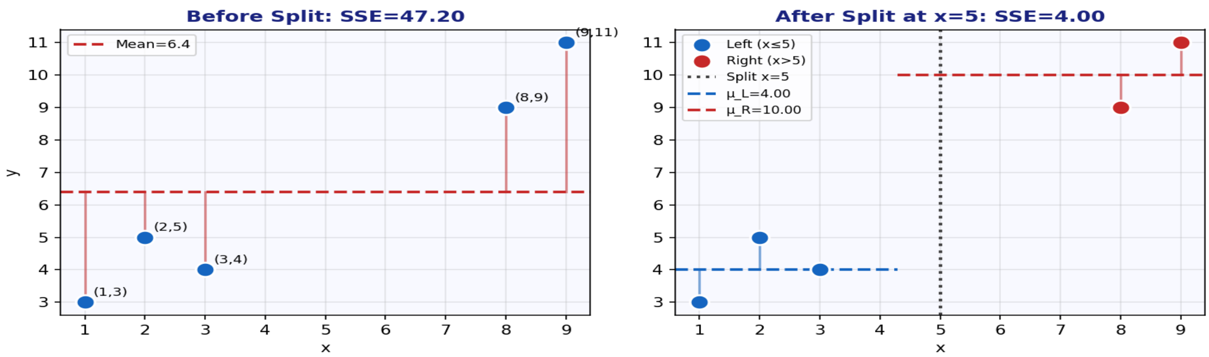
1	**SSE before splitting (all 5 points as one node):** $\bar{y} = (3+5+4+9+11)/5 = 32/5 = 6.4$ $SSE = (3-6.4)^2+(5-6.4)^2+(4-6.4)^2+(9-6.4)^2+(11-6.4)^2$ $= 11.56 + 1.96 + 5.76 + 6.76 + 21.16 = 47.20$

2	**Left child (x≤5): points (1,3),(2,5),(3,4):**
	`ȳ_L = (3+5+4)/3 = 12/3 = 4.0`
	`SSE_L = (3−4)²+(5−4)²+(4−4)² = 1+1+0 = 2.0`

3	**Right child (x>5): points (8,9),(9,11):**
	`ȳ_R = (9+11)/2 = 20/2 = 10.0`
	`SSE_R = (9−10)²+(11−10)² = 1+1 = 2.0`

4	**Total SSE after split:** `SSE_split = SSE_L + SSE_R = 2.0 + 2.0 = 4.0` **Reduction in SSE = 47.20 − 4.0 = 43.20** — an enormous improvement (91.5% reduction). This is an excellent split.

? SSE_before=47.20 | SSE_after=4.0 | Reduction=43.20 (91.5% — excellent split)



Left: all 5 points around global mean (large errors). Right: split at x=5 creates two tight groups with SSE=2 each.

Q20 — Naïve Bayes: Violated Independence Assumption

Give a realistic example where the independence assumption is clearly violated. Why does Naïve Bayes still work well in practice?

? Clear Violation Example

Text classification with word co-occurrences:

In email spam detection, the words 'free' and 'offer' are highly correlated — they tend to co-occur in the same spam emails.

NB assumes: P(free, offer | spam) = P(free|spam)·P(offer|spam), treating them as independent. But in reality, seeing 'free' makes 'offer' much more likely. The independence assumption clearly fails.

Other examples: *'credit' and 'card'; 'blood' and 'pressure' in medical notes.*

? Why NB Still Works Well

Classification, not calibration:

Even if the absolute posterior probabilities P(y|x) are wrong, the **ranking** of classes is often correct. For a binary decision, we only need to know which class score is higher — not the exact values.

Correlated features double-count evidence, but they double-count it for ALL classes similarly, so the relative comparison often still points to the right class.

Empirically, NB achieves strong performance on spam detection, document classification, and medical diagnosis despite violated assumptions.

? All 20 Answers at a Glance

Q	Answer
Q1	GDA=generative, LR=discriminative. GDA wins if Gaussian. LR wins if not.
Q2	P=0 kills product. Fix: P=(c+1)/(N+|V|) — Laplace smoothing.
Q3	Claim correct in raw space. Fix: add polynomial features $\varphi(x)$.
Q4	$P(y=1\|x_1=1,x_2=0) \approx 0.333$ (with Laplace). Predict y=0.
Q5	$z=2\sqrt{5}\approx 4.472$, $\hat{x}=[4,2]^T$, error=0 (x lies on PC1).
Q6	Bad init → K-Means++. Wrong shape → DBSCAN or GMM.
Q7	Plot SSE vs K. Elbow = optimal K. SSE always ↓ as K↑.
Q8	Trivial classifier: Recall=0. Use Precision=TP/(TP+FP), Recall=TP/(TP+FN).
Q9	Total=200. 3 PCs=62.5%. Need 4 PCs for ≥70%.
Q10	$C_1=\{3,5\}\mu=4$, $C_2=\{11,13\}\mu=12$. Converged (centroids unchanged).
Q11	Gini_parent=0.486. Weighted=0.139. Reduction=0.347 — good split.
Q12	Overfitting. Pre: max_depth/min_samples. Post: cost-complexity pruning.
Q13	$\varphi=n_1/n$; μ_0=mean(class-0 pts); μ_1=mean(class-1 pts); Σ=pooled cov.
Q14	F1_A=0.686, F1_B=0.737. Medical: Classifier B (higher Recall=0.85).
Q15	$\theta_j \leftarrow \theta_j - \eta(1/m)\Sigma(h\theta - y)x_j$. Cross-entropy convex; squared error not.
Q16	Threshold (90% var) or scree plot. Choose smaller for speed/interpretability.
Q17	GDA better if assumptions hold. LR better if violated (robust).
Q18	$P(\text{excluded})=(1-1/n)^n\approx 36.8\%$. OOB = free validation set in Random Forest.
Q19	SSE_before=47.2, SSE_after=4.0. Reduction=43.2 (91.5% — excellent).
Q20	'free'+'offer' correlated → independence violated. NB works: ranking correct.